

Playwright + pytest Command Notes

PYTHON / PLAYWRIGHT / PYTEST

A quick reference of handy Playwright and pytest snippets.

Source: These commands are based on the Udemy course [Playwright PYTHON Automation Testing - From Zero to Expert](#).

Locators

CSS locator by attribute: # for id, . for class, tag name for the element.

```
#locator css : attribute(id) # , class . , tag
```

Running tests

```
# single test in a file (headed = visible browser)
pytest -s playwrightBasics.py::test_playwright_shortcut --headed

# whole file, choosing the browser
pytest -s test_e2e_framework_web_api.py --headed --browser_name firefox
```

Markers

```
@pytest.mark.smoke # group
@pytest.mark.skip # skip test
```

Assertions & delay

```
expect(my_page.locator("h1")).to_have_text("Your Orders")
```

Filtering elements

Get an element by filter:

```
order_raw = my_page.locator("//tbody/tr").filter(has_text=order_id)
```

XPath contains text:

```
//td/button[contains(text(), 'View')]
//div[contains(@class, 'mt-4')]
```

Child window (popup)

```
with page.expect_popup() as new_page:
    page.locator(".blinkingText").first.click()
    child_page = new_page.value
```

Alert box (dialog)

```
# without lambda
def handle_dialog(dialog: Dialog):
    dialog.accept()

# page.on("dialog", handle_dialog) # without lambda
page.on("dialog", lambda dialog: dialog.accept()) # lambda example: add = lambda a, b: a + b -> print(add(2, 3))

# opens the dialog; Playwright automatically listens and confirms it
page.get_by_role("button", name="Confirm").click()
time.sleep(3)
```

Tables

```
for col_index in range(page.locator("th").count()):
    if page.locator("th").nth(col_index).filter(has_text="Price").count() > 0:
        price_col_index = col_index
        print(f"price col index : {price_col_index}")
        break

rice_row = page.locator("tr").filter(has_text="Rice")

expect(rice_row.locator("td").nth(price_col_index)).to_contain_text("37")
```

Positional selectors

```
# nth element
order_raw.locator("//td").nth(view_id_index).get_by_role("button", name="View").click()

# first element
order_raw.locator("//td").first.click()

# last element
order_raw.locator("//td").last.click()
```

API

Payloads:

```
orders_payload = {"orders": [{"country": "India", "productOrderedId": "6960eac0c941646b7a8b3e68"}]}
login_payload = {"userEmail": "rahulshetty@gmail.com", "userPassword": "Iamking@000"}
```

Utility class:

```
class APIUtils:
    def get_token(self, playwright: Playwright):
        api_request_context = playwright.request.new_context(base_url="https://rahulshettyacademy.com")

        response = api_request_context.post(
            "/api/ecom/auth/login",
            data=login_payload,
            headers={"Content-Type": "application/json"},
        )
        # print(response.status)
        # print(response.text())
        assert response.ok
        response_body = response.json()
        return response_body["token"]

    def create_order(self, playwright: Playwright):
        api_request_context = playwright.request.new_context(base_url="https://rahulshettyacademy.com")
        response = api_request_context.post(
            "/api/ecom/order/create-order",
            data=orders_payload,
            headers={
                "Authorization": self.get_token(playwright),
                "Content-Type": "application/json",
            },
        )
        return response.json()["orders"][0]
```

In test:

```
api_util = APIUtils()
print(f"token :{api_util.get_token(playwright)}")
order_id = api_util.create_order(playwright)
```

Mocking

Fulfill (server response):

```
fake_payload_order_response = {"data": [], "message": "No Orders"}

def intercept_response(route):
    route.fulfill(json=fake_payload_order_response)

page.route("https://rahulshettyacademy.com/api/ecom/order/get-orders-for-customer/*", intercept_response)
```

Continue (client request):

```
def intercept_response_not_author(route):
    # replace the call request from the browser
    route.continue_(url="https://rahulshettyacademy.com/api/ecom/order/get-orders-details?id=6a61eba885b8849b49068294")

page.route("https://rahulshettyacademy.com/api/ecom/order/get-orders-details?id=", intercept_response_not_author)
```

Inject data into context session

```
my_token = api_utils.get_token(playwright)
my_browser = playwright.chromium.launch(headless=False)
print(f"my_token :{my_token}")
my_context = my_browser.new_context()
my_page = my_context.new_page()
my_page.add_init_script(f"""localStorage.setItem('token', '{my_token}')""")
```

JSON dataset (data-driven tests)

Runs the test once per record. data/credentials/credentials.json maps a named key to each record:

```
{
  "user_a": { "userEmail": "...", "UserPassword": "..."},
  "user_b": { "userEmail": "...", "UserPassword": "..."}
}
```

Loaders (see utils/data_reader.py):

```
def load_credentials():
    with open('data/credentials/credentials.json') as json_file:
        return list(json.load(json_file).values()) # -> [ {user_a}, {user_b} ]

def load_credential_ids():
    with open('data/credentials/credentials.json') as json_file:
        return list(json.load(json_file).keys()) # -> ["user_a", "user_b"]
```

Parametrize the test, labelling each case by its key with ids=:

```
# ids= labels each case: test_e2e_api[user_a], test_e2e_api[user_b]
@pytest.mark.parametrize('user_credential', load_credentials(), ids=load_credential_ids())
# @pytest.mark.parametrize('user_credential', load_credentials(), ids=lambda user: user["userEmail"]) # label by email
↪ instead
def test_e2e_api(context_setup, user_credential):
    ...
```

Playwright UI Elements Command Reference

LOCATORS / FORM CONTROLS

Common UI element patterns using Playwright Python sync API. Prefer user-facing locators such as role, label, placeholder, text, and test id before CSS/XPath.

Recommended locators

Use locators that reflect how the user sees or accesses the control.

```
page.get_by_role("button", name="Save")
page.get_by_label("Email")
page.get_by_placeholder("Search...")
page.get_by_text("Welcome")
page.get_by_alt_text("Company logo")
page.get_by_title("Close")
page.get_by_test_id("submit-button")
page.locator("#id")
page.locator(".class-name")
page.locator("input[name='email']")
```

Text box / email / password / search

`fill()` is the normal choice for text-like inputs. An empty string clears the value.

```
page.get_by_label("Username").fill("dudu")
page.get_by_label("Email").fill("name@example.com")
page.get_by_label("Password").fill("secret")
page.get_by_role("searchbox").fill("Playwright")
page.get_by_label("Username").fill("") # clear
```

Textarea / contenteditable

`fill()` also works with and contenteditable elements.

```
page.get_by_label("Description").fill("Automation notes")
page.locator("[contenteditable='true']").fill("Editable content")
expect(page.get_by_label("Description")).to_have_value("Automation notes")
```

Button

Locate by accessible role and name, then click. `dblclick()` is available when needed.

```
page.get_by_role("button", name="Submit").click()
page.get_by_role("button", name="Refresh").dblclick()
expect(page.get_by_role("button", name="Submit")).to_be_enabled()
```

Checkbox

`check()`, `unchecked()`, and `set_checked()` explicitly control the checked state.

```
terms = page.get_by_label("I agree")
terms.check()
expect(terms).to_be_checked()
terms.uncheck()
terms.set_checked(True)
```

Radio button

Radio inputs use the same checked APIs.

```
page.get_by_label("XL").check()
expect(page.get_by_label("XL")).to_be_checked()
page.get_by_role("radio", name="Express").check()
```

Playwright UI Elements Command Reference

SELECTION / NAVIGATION CONTROLS

Patterns for native selects, custom dropdowns, tabs, menus, links, and common form-like widgets.

Native dropdown -

Use `select_option()` for a real HTML select. Select by value, label, or multiple values.

```
color = page.get_by_label("Choose a color")
color.select_option("blue")
color.select_option(label="Blue")
page.get_by_label("Colors").select_option(["red", "green", "blue"])
```

Custom dropdown / listbox

Custom dropdowns are usually clicked and then an option is selected by role.

```
page.get_by_role("combobox", name="Country").click()
page.get_by_role("option", name="Israel").click()

listbox = page.get_by_role("listbox")
listbox.get_by_role("option", name="Tel Aviv").click()
```

Autocomplete / combobox

Fill the combobox, wait for matching options, then choose one.

```
city = page.get_by_role("combobox", name="City")
city.fill("Beer")
page.get_by_role("option", name="Beer Sheva").click()
```

Date / time / datetime-local

HTML date and time controls can be filled using their expected value format.

```
page.get_by_label("Birth date").fill("2026-08-09")
page.get_by_label("Appointment time").fill("15:30")
page.get_by_label("Local time").fill("2026-08-09T15:30")
```

Number / spinbutton

Number inputs commonly expose the spinbutton role.

```
qty = page.get_by_role("spinbutton", name="Quantity")
qty.fill("5")
qty.press("ArrowUp")
expect(qty).to_have_value("6")
```

Slider

Accessible sliders can be adjusted with keyboard actions and verified with `aria-valuenow`.

```
slider = page.get_by_role("slider", name="Volume")
slider.press("Home")
slider.press("ArrowRight")
expect(slider).to_have_attribute("aria-valuenow", "1")
```

Link

Use the link role for anchors and link-like controls.

```
page.get_by_role("link", name="Orders").click()
expect(page.get_by_role("link", name="Orders")).to_have_attribute("href", "/orders")
```

Tabs

Click a tab by role and verify selection when the widget exposes aria-selected.

```
tab = page.get_by_role("tab", name="Details")
tab.click()
expect(tab).to_have_attribute("aria-selected", "true")
expect(page.get_by_role("tabpanel")).to_be_visible()
```

Menu / menu item

Open the menu, then interact with individual menu items.

```
page.get_by_role("button", name="More").click()
page.get_by_role("menuitem", name="Delete").click()
```

Playwright UI Elements Command Reference

CONTENT / COLLECTIONS / STRUCTURE

Patterns for repeated elements, tables, images, files, modals, frames, and browser-created pages.

List / repeated elements

Use `count()`, `nth()`, `first`, `last`, `filter()`, or `all()` when working with collections.

```
items = page.get_by_role("listitem")
print(items.count())
items.first.click()
items.nth(2).click()
items.last.click()

row = page.locator("tr").filter(has_text="Rice")
```

Table

Locate a row first, then query cells or controls inside that row.

```
row = page.get_by_role("row").filter(has_text="Order 123")
expect(row.get_by_role("cell").nth(2)).to_contain_text("$50")
row.get_by_role("button", name="View").click()
```

Image / icon

Images are best located by alt text. For decorative icons, use an accessible button/name when possible.

```
logo = page.get_by_alt_text("Company logo")
expect(logo).to_be_visible()
expect(logo).to_have_attribute("src", "/images/logo.png")
```

File upload

Use `set_input_files()` with a file input. Empty list clears the current selection.

```
page.get_by_label("Upload file").set_input_files("report.pdf")
page.get_by_label("Upload files").set_input_files(["a.txt", "b.txt"])
page.get_by_label("Upload file").set_input_files([])
```

Dynamic file chooser

When the file input is created only after clicking, wait for the chooser.

```
with page.expect_file_chooser() as fc_info:
    page.get_by_role("button", name="Upload").click()

file_chooser = fc_info.value
file_chooser.set_files("report.pdf")
```

Modal / dialog in the DOM

Scope locators to the dialog to avoid matching controls behind it.

```
dialog = page.get_by_role("dialog")
expect(dialog).to_be_visible()
dialog.get_by_role("button", name="Confirm").click()
```

iframe

Use `frame_locator()` to find and interact with controls inside an `iframe`.

```
frame = page.frame_locator("iframe[name='login']")
frame.get_by_label("User Name").fill("John")
frame.get_by_role("button", name="Sign in").click()
```

Popup / new tab

Wrap the click with `expect_popup()` and then use the returned `Page`.

```
with page.expect_popup() as popup_info:
    page.get_by_role("link", name="Open report").click()

popup = popup_info.value
popup.wait_for_load_state()
expect(popup).to_have_title("Report")
```


Playwright UI Elements Command Reference

INTERACTION / MOUSE / KEYBOARD

Commands for dialogs, hover, context menus, keyboard events, dragging, scrolling, and special interaction cases.

Browser alert / confirm / prompt

Browser dialogs are handled through the dialog event.

```
page.on("dialog", lambda dialog: dialog.accept())
page.get_by_role("button", name="Confirm").click()

page.on("dialog", lambda dialog: dialog.dismiss())

def handle_prompt(dialog):
    dialog.accept("My answer")
page.once("dialog", handle_prompt)
```

Hover / tooltip

Hover the trigger, then verify the tooltip or hover content.

```
page.get_by_role("button", name="Help").hover()
expect(page.get_by_role("tooltip")).to_be_visible()
```

Right click / double click

click() accepts a mouse button; dblclick() performs a double-click.

```
page.get_by_text("File").click(button="right")
page.get_by_text("Open").dblclick()
```

Keyboard

press() handles special keys and shortcuts. press_sequentially() is for apps that require per-key events.

```
search = page.get_by_role("searchbox")
search.fill("playwright")
search.press("Enter")

page.keyboard.press("Control+A")
page.keyboard.press("Escape")
search.press_sequentially("hello", delay=100)
```

Focus / blur

Use focus() when the application behavior depends on focus events.

```
field = page.get_by_label("Password")
field.focus()
expect(field).to_be_focused()
field.blur()
```

Drag and drop

drag_to() is the high-level drag-and-drop API.

```
source = page.locator("#source")
target = page.locator("#target")
source.drag_to(target)
```

Scroll

Scroll a specific element into view, or use the mouse wheel for page-level scrolling.

```
target = page.get_by_text("Bottom section")
target.scroll_into_view_if_needed()
page.mouse.wheel(0, 800)
```

Canvas / coordinate-based UI

For a canvas or UI without useful DOM semantics, use mouse coordinates relative to a known element.

```
canvas = page.locator("canvas")
box = canvas.bounding_box()
if box:
    page.mouse.click(box["x"] + 100, box["y"] + 80)
```

Shadow DOM

Playwright locators work through open Shadow DOM in normal locator usage. Prefer locator APIs over XPath.

```
page.get_by_role("button", name="Settings").click()
page.locator("my-component").get_by_text("Advanced").click()
```

Playwright UI Elements Command Reference

STATE / ASSERTIONS / VALUE READING

Quick commands for checking whether an element is ready, reading values, and verifying UI behavior.

Visibility / enabled / editable

Use web-first assertions when you need to verify an element state.

```
button = page.get_by_role("button", name="Save")
expect(button).to_be_visible()
expect(button).to_be_enabled()

field = page.get_by_label("Name")
expect(field).to_be_editable()
expect(page.get_by_text("Loading")).to_be_hidden()
```

Text / value / attribute

Use assertions for test checks; direct reads are useful when you need the value in Python.

```
title = page.get_by_role("heading").inner_text()
value = page.get_by_label("Email").input_value()
href = page.get_by_role("link", name="Home").get_attribute("href")

expect(page.get_by_role("heading")).to_have_text("Dashboard")
expect(page.get_by_label("Email")).to_have_value("name@example.com")
```

Checked state

Prefer `to_be_checked()` for assertions; `is_checked()` returns a Python boolean.

```
checkbox = page.get_by_label("Subscribe")
expect(checkbox).to_be_checked()
print(checkbox.is_checked())
```

Element screenshot

Capture only a specific element when debugging.

```
page.get_by_role("dialog").screenshot(path="dialog.png")
page.locator("#chart").screenshot(path="chart.png")
```

Wait for an element state

Locators auto-wait for actions. `wait_for()` is useful when you explicitly need a state transition.

```
spinner = page.get_by_test_id("spinner")
spinner.wait_for(state="visible")
spinner.wait_for(state="hidden")
```

Useful positional and filter patterns

Use these when a page contains multiple matching elements.

```
cards = page.locator(".card")
cards.filter(has_text="Premium").get_by_role("button", name="Buy").click()

page.get_by_role("button").first.click()
page.get_by_role("button").nth(2).click()
page.get_by_role("button").last.click()
```

General pattern

A stable test usually follows: locate -> act -> assert.

```
username = page.get_by_label("User Name")
username.fill("John")

page.get_by_role("button", name="Sign in").click()

expect(page.get_by_text("Welcome, John!")).to_be_visible()
```

API note: Prefer locator-based APIs and web-first assertions. The older `locator.type()` method is deprecated; use `fill()` for normal text entry and `press_sequentially()` only when per-key handling is required.

Reference basis: [official Playwright Python documentation for Locators, Actions, Frames, and Locator API](#).

Additional pytest Data-Driven Patterns

MISSING ITEMS FOUND IN YOUR ADDITIONAL NOTES

The earlier PDF already includes the Playwright run commands, markers, assertions, filtered locators, XPath examples, dialog handling, positional selectors, API utilities, API mocking, and a JSON-based parametrize example. The following patterns were not yet included.

Session fixture using request.param

Runnable form of your fixture-based pattern. The parameter is passed through the fixture with indirect=True.

```
@pytest.fixture(scope="session")
def user_credential_fxtr(request):
    # request is pytest's built-in FixtureRequest fixture
    return request.param

with open("playwright_framework/data/credentials.json") as json_file:
    test_data = json.load(json_file)
    user_list = test_data["user_credentials"]

@pytest.mark.parametrize(
    "user_credential_fxtr",
    user_list,
    indirect=True,
)
def test_e2e_api(playwright: Playwright, user_credential_fxtr):
    # use user_credential_fxtr in the test
    ...
```

Loader function with a custom parameter ID

Your second pattern loads the user list through a helper and names each test case by userEmail.

```
def load_credentials():
    with open("playwright_framework/data/credentials.json") as json_file:
        test_data = json.load(json_file)
        user_list = test_data["user_credentials"]
        return user_list

def param_id_name(user):
    return user["userEmail"]

@pytest.mark.parametrize(
    "user_credentials_params",
    load_credentials(),
    ids=param_id_name,
)
def test_e2e_api(playwright: Playwright, user_credentials_params):
    ...
```

Same IDs pattern using lambda

Equivalent compact form from your notes.

```
@pytest.mark.parametrize(
    "user_credentials_params",
    load_credentials(),
    ids=lambda user: user["userEmail"],
)
def test_e2e_api(playwright: Playwright, user_credentials_params):
    ...
```

Note: Your pasted fixture snippet used different names (user_credential_fxtr vs user_credentials_fxtr). The PDF uses one consistent fixture name so the example is runnable. It also adds indirect=True because the fixture reads request.param.

Source basis: your additional notes. The request.param/indirect behavior was checked against the official pytest documentation.

XPath Quick Reference for Playwright

BASIC XPATH / TEXT / ATTRIBUTES / RELATIONSHIPS

Use XPath when a DOM relationship is easier to express with an axis. In Playwright Python you can use `page.locator("xpath=...")` or simply start the selector with `//`.

Basic element and attribute selectors

Find elements by tag, id, class, or any attribute.

```
//button
//input[@id='username']
//input[@name='email']
//div[@class='card']
//div[contains(@class, 'card')]
//input[@type='text' and @name='username']

page.locator("//button").click()
page.locator("xpath=//input[@id='username']").fill("John")
```

Text matching

Use `text()`, `normalize-space()`, `contains()`, and `starts-with()` for text-based matching.

```
//button[text()='Save']
//button[normalize-space()='Save']
//button[contains(text(), 'Save')]
//div[contains(normalize-space(), 'Order created')]
//a[starts-with(normalize-space(), 'View')]

page.locator("//button[normalize-space()='Save']").click()
```

Child and descendant

A single slash selects a direct child; `//` searches descendants at any depth.

```
//form/input
//form//input
//div[@id='login']//button
//table//tbody//tr

# Playwright
page.locator("//form//input[@name='email']").fill("a@b.com")
```

Parent

Move from an element to its direct parent using `parent::` or `..`

```
//span[text()='Price']/parent::div
//span[text()='Price']/..

# From an existing Playwright locator:
price = page.get_by_text("Price")
parent = price.locator("xpath=..")
```

Ancestor

Move upward to the nearest matching ancestor, useful for cards, rows, forms, and containers.

```
//button[text()='Buy']/ancestor::div[contains(@class, 'card')][1]
//input[@name='email']/ancestor::form
//span[text()='Order 123']/ancestor::tr

row = page.locator("//span[text()='Order 123']/ancestor::tr")
row.locator("./button[normalize-space()='View']").click()
```

Following sibling

Select a sibling after the current element. Add [1] for the closest matching sibling.

```
//label[text()='Email']/following-sibling::input
//label[text()='Email']/following-sibling::input[1]
//td[text()='Rice']/following-sibling::td[1]

page.locator(
  "//label[normalize-space()='Email']/following-sibling::input[1]"
).fill("user@example.com")
```

Preceding sibling

Select a sibling before the current element.

```
//button[text()='Delete']/preceding-sibling::span
//td[text()='$50']/preceding-sibling::td[1]
//input[@name='password']/preceding-sibling::label

label = page.locator(
  "//input[@name='password']/preceding-sibling::label"
)
```

XPath Quick Reference for Playwright

AXES / POSITION / COMPLEX PRACTICAL EXAMPLES

These examples cover non-sibling relationships, indexing, union, and common table/form scenarios.

following::

Select matching elements that appear later in the document, not only siblings.

```
//label[text()='Username']/following::input[1]
//h2[text()='Payment']/following::button[1]

page.locator(
  "//label[normalize-space()='Username']/following::input[1]"
).fill("John")
```

preceding::

Select matching elements that appear earlier in the document.

```
//button[text()='Submit']/preceding::input[1]
//div[text()='Total']/preceding::span[1]

previous_input = page.locator(
  "//button[normalize-space()='Submit']/preceding::input[1]"
)
```

Explicit axes: child / descendant

Axis syntax can make the relationship more readable when teaching or debugging XPath.

```
//form/child::input
//form/descendant::input
//table/descendant::button[normalize-space()='View']
```

Index and position

XPath indexing is 1-based. Parentheses apply the index to the full result set.

```
(//button)[1]
(//button)[2]
(//button)[last()]
//ul/li[1]
//ul/li[last()]
//ul/li[position()=2]
//ul/li[position() < 4]

page.locator("//button[normalize-space()='Buy']")[2].click()
```

Multiple conditions: and / or

Combine attribute, text, and state conditions.

```
//input[@type='text' and @name='username']
//button[@type='submit' and not(@disabled)]
//input[@name='email' or @name='username']
//div[contains(@class,'card') and contains(., 'Premium')]
```

XPath union: |

Return matches from either XPath expression.

```
//button[text()='Save'] | //button[text()='Submit']

page.locator(
  "//span[contains(@class,'spinner')] | //div[@id='confirmation']"
).wait_for()
```


Relative XPath inside an existing locator

Prefix with `./` to stay inside the current row/card/container.

```
row = page.locator("//tr[td[normalize-space()='Rice']]")
price = row.locator("./td[contains(@class,'price')]")
view = row.locator("./button[normalize-space()='View']")
view.click()
```

Practical table examples

Locate a row from cell text, then target another cell or button in the same row.

```
# Row containing Rice
//tr[td[normalize-space()='Rice']]

# Price cell in the Rice row
//tr[td[normalize-space()='Rice']]/td[contains(@class,'price')]

# View button in Order 123 row
//tr[td[normalize-space()='Order 123']]/button[normalize-space()='View']

order_row = page.locator(
    "//tr[td[normalize-space()='Order 123']]"
)
order_row.locator("./button[normalize-space()='View']").click()
```

Practical label -> input examples

Useful when the label and input are related in the DOM but lack a stable id/for relationship.

```
# Input immediately after label
//label[normalize-space()='Email']/following-sibling::input[1]

# First input anywhere after label
//label[normalize-space()='Email']/following::input[1]

# Input inside the same parent container
//label[normalize-space()='Email']/parent::*//input
```

Playwright tip: XPath is supported, but `role`, `label`, `text` and `test-id` locators are generally more resilient when available. XPath also does not pierce shadow roots.

Reference basis: [official Playwright Python locator documentation](#) and [standard XPath axis syntax](#).

pytest Fixture Reference

ALL FIXTURE SCOPES

pytest supports five standard fixture scopes. Scope controls how long a fixture instance is reused and when its teardown runs. function is the default.

Scope	Decorator	Lifetime / reuse	Typical use
function	@pytest.fixture() or scope="function"	One instance per test; teardown after that test.	Fresh page/context, isolated test data.
class	@pytest.fixture(scope="class")	Shared by tests in one class; teardown after its last test.	Shared setup for methods in one test class.
module	@pytest.fixture(scope="module")	Shared by tests in one module; teardown after its last test.	Expensive module-level setup/API client.
package	@pytest.fixture(scope="package")	Shared in the package where defined, including subpackages/directories.	Setup reused across a test package.
session	@pytest.fixture(scope="session")	Shared for the entire pytest run; teardown at session end.	Global config, expensive engine/server/client.

All five scope declarations

The empty decorator and scope="function" are equivalent.

```
@pytest.fixture()
def function_fixture():
    return "new for each test"

@pytest.fixture(scope="class")
def class_fixture():
    return "shared within one class"

@pytest.fixture(scope="module")
def module_fixture():
    return "shared within one test module"

@pytest.fixture(scope="package")
def package_fixture():
    return "shared within one package"

@pytest.fixture(scope="session")
def session_fixture():
    return "shared for the whole pytest session"
```

Broadest -> narrowest: session -> package -> module -> class -> function. Higher-scoped fixtures are set up before lower-scoped fixtures for a test request.

pytest Fixture Reference

SETUP / TEARDOWN / AUTOUSE / DEPENDENCIES

Fixtures can provide data, manage cleanup after yield, run automatically, and depend on other fixtures.

return fixture - setup only

Use return when no teardown is needed.

```
@pytest.fixture
def user_data():
    return {"username": "dudu", "role": "admin"}

def test_user(user_data):
    assert user_data["username"] == "dudu"
```

yield fixture - setup + teardown

Code before yield is setup. Code after yield is teardown.

```
@pytest.fixture(scope="function")
def browser_page(browser):
    page = browser.new_page()
    yield page
    page.close() # teardown
```

autouse=True

Runs automatically for every test that can see the fixture.

```
@pytest.fixture(autouse=True)
def reset_state():
    print("setup")
    yield
    print("cleanup")

def test_example():
    pass # reset_state still runs
```

Fixture depends on another fixture

Request another fixture by adding its name as an argument.

```
@pytest.fixture(scope="session")
def api_client():
    return APIClient()

@pytest.fixture(scope="function")
def logged_in_user(api_client):
    user = api_client.create_user()
    yield user
    api_client.delete_user(user.id)
```

@pytest.mark.usefixtures

Useful when only the fixture's side effects are needed.

```
@pytest.mark.usefixtures("reset_state")
class TestOrders:
    def test_create_order(self):
        ...

@pytest.mark.usefixtures("reset_state", "seed_data")
def test_checkout():
    ...
```

pytest Fixture Reference

PARAMETRIZATION / DECORATOR OPTIONS

The `@pytest.fixture` decorator also supports `params`, `ids`, `autouse`, `name`, and a callable scope.

Scope dependency: a narrow fixture can use an equal or broader-scoped fixture (for example function -> session). A broader-scoped fixture should not depend on a narrower-scoped fixture.

params + request.param

Every test using the fixture is repeated for each parameter value.

```
@pytest.fixture(
    params=["chromium", "firefox", "webkit"],
    ids=["Chrome", "Firefox", "WebKit"],
)
def browser_name(request):
    return request.param

def test_homepage(browser_name):
    print(browser_name)
```

scope + params

Scope and parametrization can be combined.

```
@pytest.fixture(
    scope="module",
    params=["qa", "staging"],
    ids=["QA", "STAGE"],
)
def environment(request):
    yield request.param
```

name= - fixture alias

Expose the fixture to tests under a different name.

```
@pytest.fixture(name="credentials")
def fixture_credentials():
    return {"user": "test_user", "password": "secret"}

def test_login(credentials):
    ...
```

Common @pytest.fixture(...) options

Quick decorator reference.

```
@pytest.fixture(
    scope="function", # function | class | module | package | session | callable
    params=[...],     # repeat tests for fixture values
    autouse=False,    # True = automatic where visible
    ids=[...],        # readable IDs for params
    name="my_fixture", # public fixture name
)
def fixture_name(request):
    ...
```

pytest Fixture Reference

DYNAMIC SCOPE / PLAYWRIGHT PATTERN

Dynamic scope is useful when the same fixture should be function-scoped in one run and session-scoped in another.

Dynamic scope

scope can be a callable receiving fixture_name and config. It must return a valid scope string.

```
def determine_scope(fixture_name, config):
    if config.getoption("--keep-browser", default=False):
        return "session"
    return "function"

@pytest.fixture(scope=determine_scope)
def browser_resource():
    resource = create_resource()
    yield resource
    resource.close()
```

Playwright-style scope pattern

Keep expensive engine/browser objects broad, while context/page state is recreated for each test.

```
@pytest.fixture(scope="session")
def playwright_engine():
    with sync_playwright() as p:
        yield p

@pytest.fixture(scope="session")
def browser(playwright_engine):
    browser = playwright_engine.chromium.launch()
    yield browser
    browser.close()

@pytest.fixture(scope="function")
def page(browser):
    context = browser.new_context()
    page = context.new_page()
    yield page
    context.close()
```

Remember: function is the default scope, so `@pytest.fixture` and `@pytest.fixture(scope="function")` have the same lifetime.

Reference basis: official pytest documentation for fixture scopes, dynamic scope, yield teardown, autouse, parametrization, usefixtures, and fixture decorator parameters.